Milestone 1 – Coding Mastery Guide (NumPy · Pandas · FastAPI)

Oct 24, 2026 · @Prem

1. Read this first: exam pattern and how to score full marks

Milestone 1 (Day 12, 29 Oct 2026) tests Days 1–11: Python, NumPy, Pandas, SQL, FastAPI, Pydantic, ORM, security and testing. You told me the coding paper has three question types, so this guide prepares all of them:

Pattern	What it tests	Where in this guide
NumPy + Pandas mix	Arrays, broadcasting, boolean masks, DataFrame cleaning, groupby	Sections 3, 4
Pandas pipeline (30 marks)	Six small functions: load, clean, merge, derive, aggregate, finalize	Section 5
FastAPI + Pydantic + SQLAlchemy (30 marks)	Parent-child CRUD API with validation, filter, sort, tests	Section 6

I am assuming the two 30-mark questions are the Pandas pipeline and the FastAPI API, as in your practice pack. If the paper is different, the skills below still cover it.

How graders almost certainly mark these (pytest-style, automated):

- Exact function names and parameter names. One typo = that whole function fails.
- Exact output columns, in exact order, with a clean 0..n-1 index.
- Exact status codes (201, 200, 404, 422), not "close enough".
- No printing inside functions. No running the pipeline or server on import.
- No hard-coded values. Hidden test data will differ from the sample CSV.

The five-step routine for every coding question:

1. Read the spec twice. Copy every function name, parameter name and column name into a checklist.
2. Write the skeleton first (all function signatures, `return` placeholders, `if __name__` block).
3. Fill functions one at a time. Run after each one with `print(df.head())` in a scratch cell, not in the function.

4. Test the edge cases from the spec's "Test checklist" (missing, duplicate, unmatched, rounding, order).
5. Final pass: remove debug prints, check column order, check `if __name__ == '__main__'`.

Time split suggestion for a ~3-hour paper: NumPy/Pandas mix 40 min, Pandas pipeline 45 min, FastAPI 75 min, review 20 min. Do the question you are strongest at first to bank marks.

How to study with limited time (your constraint): one hour of self-study a day is enough if you type the solutions from a blank file, not read them. Do each worked solution in Sections 5 and 6 three times: once with this guide open, once with only the spec, once with a timer.

2. Python foundations the coding questions silently depend on

Most lost marks are not NumPy or FastAPI gaps. They are Python slips. Be able to do each item below from memory.

Functions and structure

- Write a function with a docstring, type hints and a `return` on every path. A function that forgets `return` gives `None` and the next pipeline step crashes.
- `*args` (tuple of extras), `**kwargs` (dict of extras), default arguments, keyword-only arguments, variable scope (local vs global).
- Never use a mutable default (`def f(x=[])`). Use `None` and create the list inside.
- `if __name__ == '__main__':` runs only when the file is executed, not when it is imported. Every pipeline and server file needs it.

Data structures

- List vs tuple vs set vs dict: mutability, ordering, duplicates, lookup cost.
- List, dict and set comprehensions, with a condition: `{k: v for k, v in d.items() if v > 0}`.
- `sorted(items, key=lambda x: x['fee'])`, `reverse=True`, stable sorting.
- `dict.get(key, default)` vs `d[key]` (KeyError). `enumerate`, `zip`, `any`, `all`.

OOP (needed for SQLAlchemy models and Pydantic)

- Class, `__init__`, attributes, methods, inheritance (`class Consultant(Base)`, `class X(BaseModel)`), `super()`.
- Class attribute vs instance attribute. `__repr__` vs `__str__`.

Errors, files, modules

- `try / except / else / finally`; catch specific exceptions; `raise ValueError('msg')`.
- `with open(path) as f:` and `json.load`, `csv.DictReader`. Relative paths like `data/file.csv`, not `C:/Users/...`.
- `import` styles, `from x import y`, a package with `__init__.py`, `requirements.txt`.

Iterators, generators, decorators

- `yield`, generator expressions, `next()`. A decorator is a function that takes a function and returns a wrapper. FastAPI's `@app.get(...)` is a decorator, so you must be able to read one.

Python slips that cost marks

Slip	Fix
`=` vs `==` in conditions	`==` compares
Integer division surprises: `7 / 2 = 3.5`, `7 // 2 = 3`	Know both
Modifying a list while looping over it	Loop over a copy or build a new list
`is` vs `==`	Use `==` for values; `is None` for None
Indentation error after copy-paste	Re-run the file after every paste
String `"10"` vs int `10`	Convert explicitly

3. NumPy: everything you can be asked

Core idea: a NumPy array is a fixed-type block of numbers. Operations apply to the whole array at once (vectorised), with no Python loop.

3.1 Creating arrays

```
import numpy as np
a = np.array([1, 2, 3])                 # from list
z = np.zeros((2, 3)); o = np.ones((2, 3)); f = np.full((2, 2), 7)
r = np.arange(0, 10, 2)                 # [0 2 4 6 8]  (stop excluded)
l = np.linspace(0, 1, 5)                # 5 points, stop INCLUDED
i = np.eye(3)                           # identity matrix
rng = np.random.default_rng(42); x = rng.integers(1, 100, size=(3, 4))
```

3.2 Attributes you must know: `a.shape`, `a.ndim`, `a.size`, `a.dtype`, `a.itemsize`. Know that `shape` of a 1-D array of 3 is `(3,)`, not `(3, 1)`.

3.3 Indexing and slicing

```
m = np.arange(12).reshape(3, 4)
m[1, 2]        # single element
m[:, 1]        # column 1
m[0:2, ::2]    # first 2 rows, every 2nd column
m[-1]          # last row
m[[0, 2]]      # fancy indexing: rows 0 and 2 (returns a COPY)
```

3.4 Boolean masks (asked very often)

```
scores = np.array([45, 80, 62, 91, 30])
scores[scores > 60]                    # [80 62 91]
np.where(scores >= 50, 'pass', 'fail') # element-wise if/else
mask = (scores > 40) & (scores < 90)   # use & | ~ with brackets, never
and/or/not
count = (scores > 60).sum()            # True counts as 1
```

3.5 Broadcasting rule: compare shapes from the right. Two dimensions are compatible if they are equal or one of them is 1.

```
prices = np.array([[100, 200], [300, 400]])   # shape (2, 2)
discount = np.array([0.1, 0.2])               # shape (2,)  -> applied per
column
net = prices * (1 - discount)
row_disc = np.array([[0.1], [0.2]])           # shape (2, 1) -> applied per
row
```

3.6 Aggregations and axis

- `axis=0` collapses rows (gives one value per column). `axis=1` collapses columns (one value per row).
- `sum, mean, median, std, var, min, max, argmin, argmax, cumsum, prod`.
- `np.percentile(a, 75)`, `np.unique(a, return_counts=True)`, `np.sort`, `np.argsort`.

3.7 Reshaping and combining: `reshape`, `flatten` (copy) vs `ravel` (view when possible), `.T`, `np.concatenate`, `np.vstack`, `np.hstack`, `np.split`. Use `-1` to let NumPy infer one dimension: `a.reshape(-1, 1)`.

3.8 Maths and linear algebra: `np.dot`, `@`, `np.matmul` (matrix product) vs `*` (element-wise). `np.linalg.inv`, `np.linalg.det`, `np.round(a, 2)`.

3.9 Missing values: `np.nan` is a float. `np.isnan(a)`, `np.nanmean(a)`, `np.nansum(a)`. `np.nan == np.nan` is `False`.

Where people lose marks in NumPy

- Using `and` / `or` on arrays instead of `&` / `|`. This raises "truth value of an array is ambiguous".
- Forgetting the axis: `mean()` without `axis` gives one number for the whole array.
- Slices are views: changing a slice changes the original. Use `.copy()` when asked for an independent array.
- `arange` excludes the stop value; `linspace` includes it.
- Mixing `*` and `@` for matrix multiplication.
- Integer arrays silently truncate: assigning `2.7` into an int array stores `2`.
- Looping with `for` when the question says "vectorised" or "without loops".

NumPy practice questions (write each as a function that returns a value, no hard-coding)

1. Given a 2-D array of marks (students × subjects), return the average mark per student, rounded to 2 decimals.
2. Return the average per subject, and the index of the subject with the highest average.
3. Return only the rows where every subject mark is at least 40.
4. Replace all marks below 35 with 35 (grace marks) without a loop, returning a new array.
5. Normalise each column to the 0–1 range (min-max) using broadcasting.
6. Given prices and a per-item discount array, return the net price array rounded to 2 decimals.
7. Given an array containing `np.nan`, replace NaN with the column mean.
8. Return the top 3 values and their indices from a 1-D array.
9. Count how many values are above the overall mean.
10. Given a 1-D array, return a 2-D array with 3 columns (what if the length is not divisible by 3?).
11. Return the running total (cumulative sum) of daily sales and the day on which it first crosses 1000.
12. Given two arrays `a` and `b`, return the dot product and explain why `a * b` is different.
13. Standardise values: `(x - mean) / std` and verify the new mean is about 0.
14. Convert a float array of fees to integers safely (rounding first) and check the dtype.
15. Return unique values and how often each occurs, sorted by frequency descending.

4. Pandas: everything you can be asked

4.1 Loading and inspecting

```
import pandas as pd
df = pd.read_csv('data/file.csv')                          # all columns
df = pd.read_csv('data/file.csv', usecols=['a', 'b'])      # only these
columns, in file order
df.head(); df.info(); df.describe(); df.shape; df.dtypes; df.columns.tolist()
```

4.2 Selecting

```
df['col']                    # Series
df[['a', 'b']]               # DataFrame (double brackets)
df.loc[df['fee'] > 100, ['name', 'fee']]   # label/mask based
df.iloc[0:3, 0:2]            # position based
df[(df['fee'] > 100) & (df['cat'] == 'AI')]   # & | ~ with brackets
df[df['cat'].isin(['AI', 'Data'])]
df.query('fee > 100 and cat == "AI"')
```

4.3 Cleaning (the heart of Milestone questions)

```
df.isna().sum()                          # missing per column
df.dropna(subset=['email'])              # drop rows missing email only
df.fillna({'fee': 0})                    # fill specific columns
df.drop_duplicates(subset=['id'], keep='first')   # keep first occurrence
df.drop(columns=['date'])
df.rename(columns={'old': 'new'})
df['fee'] = df['fee'].astype(float)
df['name'] = df['name'].str.strip().str.title()
pd.to_numeric(df['fee'], errors='coerce')   # bad text -> NaN
```

Blank strings are not NaN. If the spec says "missing", also handle `''` using `df['email'].replace('', pd.NA)` before `dropna`.

4.4 Creating columns (vectorised)

```
df['net'] = (df['fee'] * (1 - df['disc'] / 100)).round(2)
df['grade'] = np.where(df['score'] >= 50, 'Pass', 'Fail')
df['band'] = pd.cut(df['fee'], bins=[0, 1000, 5000, np.inf], labels=['Low',
'Mid', 'High'])
df = df.assign(net=lambda d: d['fee'] * 0.9)    # returns a new DataFrame
```

4.5 Merging

```
pd.merge(left, right, on='course_id', how='inner')   # only matching keys
# how='left' keeps all left rows; 'right'; 'outer'
```

If key column names differ: `left_on='a', right_on='b'`. If both frames have a same-named non-key column, Pandas adds `_x` and `_y` suffixes. Select the needed columns of the reference frame **before** merging if the spec lists exact columns.

4.6 GroupBy and aggregation

```
out = (df.groupby('course_name', as_index=False)
         .agg(avg_net_quoted_fee=('net_quoted_fee', 'mean'),
              enquiry_count=('enquiry_id', 'count')))
```

This **named aggregation** form gives you exact output column names and a clean index in one step. Also know `size()` (counts rows including NaN) vs `count()` (counts non-NaN), `nunique()`, `sum`, `max`, `min`, `median`, and `pivot_table`.

4.7 Sorting, index, rounding, types

```
df.sort_values('course_name')                  # ascending by default
df.sort_values(['cat', 'fee'], ascending=[True, False])
df.reset_index(drop=True)                      # clean 0..n-1 index, no old
index column
df['avg'] = df['avg'].round(2)
df = df[['course_name', 'avg_net_quoted_fee', 'enquiry_count']]   # force
column order
```

4.8 NumPy + Pandas mix (the mixed-question pattern)

Typical task: load a CSV, clean it, then use NumPy for a calculation or conditional column.

```
import numpy as np, pandas as pd

def add_grade(df):
    out = df.copy()
    out['total'] = out[['maths', 'physics', 'chem']].sum(axis=1)
    out['avg'] = out['total'] / 3
    out['grade'] = np.select(
        [out['avg'] >= 75, out['avg'] >= 50],
        ['A', 'B'], default='C')
    return out

def z_score(df, col):
    arr = df[col].to_numpy(dtype=float)
    return (arr - arr.mean()) / arr.std()
```

Know: `Series.to_numpy()`, `np.where`, `np.select` (several conditions), `np.nan`, `df.values`, `pd.Series(np_array, index=df.index)`. Note NumPy `std()` uses population std (ddof=0) while Pandas `.std()` uses sample std (ddof=1). That mismatch is

a favourite trap.

Where people lose marks in Pandas

- Modifying the input DataFrame in place. Start every function with `out = df.copy()` or use methods that return new frames.
- Forgetting `reset_index(drop=True)` after filtering, sorting or grouping.
- `keep='last'` (or the default confusion) when the spec says keep first. `keep='first'` is the default but write it explicitly.
- `dropna()` with no `subset` removes rows with a missing value in **any** column, not just email.
- Chained assignment (`df[df.a > 1]['b'] = 0`). Use `df.loc[mask, 'b'] = 0`.
- Rounding before the calculation is finished, instead of rounding the final value.
- `groupby` without `as_index=False` leaves the key in the index, and the key column disappears from the output.
- Output columns in the wrong order, or an extra column such as `enquiry_date` left in.
- Using `df['col'] == np.nan` to find missing values. Use `isna()`.
- `and` / `or` instead of `&` / `|`.
- Row-by-row `iterrows()` loops when the spec says vectorised.

Pandas practice questions

1. Load only 5 named columns from a CSV, in the order the spec lists (not file order).
2. Remove rows with missing `email` and rows with blank-string `email`.
3. Keep the first of duplicate IDs, then explain what changes if you keep last.
4. Inner-join two tables, and report how many rows were dropped and why.
5. Add `net = price * (1 - disc/100)` rounded to 2 decimals without modifying the input.
6. Return per-category average, count and max, with exact column names, sorted descending by average, with a clean index.
7. Return the top 2 categories by total revenue.
8. Add a `band` column (Low/Mid/High) from fee ranges using `pd.cut` and, separately, `np.select`.
9. Fill missing `discount_pct` with 0 only for category 'AI' rows.
10. Count how many distinct students enquired per course (`nunique`).
11. Pivot: rows = category, columns = discount value, values = average net fee.
12. A function receives an empty DataFrame. Make sure aggregation still returns the right columns.

5. The Pandas pipeline question (30 marks): full worked solution

Exercises 1 and 2 in your pack are the same pattern with different names: **load → clean → merge → derive → aggregate → finalize**. Learn the pattern once and you can solve any variant (enquiries, service requests, orders, tickets, employees). This solution was run against your sample CSV and passed a test file covering every point in the pack's checklist.

Mapping between the two exercises (so you can swap names fast)

Step	Exercise 1	Exercise 2
Load	`load_student_enquiries`	`load_service_requests`
Clean	`clean_enquiry_data`	`clean_service_requests`
Merge	`merge_with_courses`	`merge_with_services`
Key	`enquiry_id`, `course_id`	`request_id`, `service_id`
Reference columns	`course_name, category, duration_months, standard_fee`	`service_name, category, duration_days, standard_fee`
Aggregate	`aggregate_by_course`	`aggregate_by_service`
Count column	`enquiry_count`	`request_count`

```
import pandas as pd


def load_student_enquiries(file_path):
    cols = ["enquiry_id", "course_id", "student_name", "email",
            "phone", "quoted_fee", "discount_pct"]
    return pd.read_csv(file_path, usecols=cols)[cols]   # [cols] forces the
exact order


def clean_enquiry_data(df):
    out = df.copy()                                     # never mutate
the input
    out = out.dropna(subset=["email"])                  # only email
decides
    out = out.drop_duplicates(subset=["enquiry_id"], keep="first")
    return out.reset_index(drop=True)
```

```
def merge_with_courses(enquiries_df, ref_file_path):
    courses = pd.read_csv(ref_file_path)
    keep = ["course_id", "course_name", "category",
            "duration_months", "standard_fee"]
    merged = enquiries_df.merge(courses[keep], on="course_id", how="inner")
    return merged.reset_index(drop=True)


def add_net_quoted_fee(df):
    out = df.copy()
    out["net_quoted_fee"] = (
        out["quoted_fee"] * (1 - out["discount_pct"] / 100)
    ).round(2)
    return out


def aggregate_by_course(df):
    return (
        df.groupby("course_name", as_index=False)
        .agg(avg_net_quoted_fee=("net_quoted_fee", "mean"),
             enquiry_count=("enquiry_id", "count"))
        .reset_index(drop=True)
    )


def finalize_summary(df):
    out = df[["course_name", "avg_net_quoted_fee", "enquiry_count"]].copy()
    out["avg_net_quoted_fee"] = out["avg_net_quoted_fee"].round(2)
    return out.sort_values("course_name").reset_index(drop=True)


if __name__ == "__main__":
    df = load_student_enquiries("data/training_enquiries.csv")
    df = clean_enquiry_data(df)
    df = merge_with_courses(df, "data/training_courses.csv")
    df = add_net_quoted_fee(df)
    df = aggregate_by_course(df)
    df = finalize_summary(df)
    print(df)
```

Output on your sample data (check yours matches)

course_name	avg_net_quoted_fee	enquiry_count
Cloud Fundamentals	20000.0	1

course_name	avg_net_quoted_fee	enquiry_count
Data Analytics	27100.0	2
Generative AI Basics	20900.0	1
Python Full Stack	22800.0	1
Software Testing	19000.0	1

Why these numbers: Meera (missing email) is gone, the second 3002 row is gone (first kept, so Python Full Stack is 24000 at 5% = 22800), and course 99 is dropped by the inner join. Data Analytics = (27000 + 27200) / 2 = 27100.

Why each line exists (the questions an interviewer or examiner asks)

- `usecols=cols` then `[cols]`: `usecols` alone returns file order, not your order. The second step guarantees the order.
- `dropna(subset=['email'])`: without `subset`, any NaN anywhere kills the row.
- `keep='first'`: the duplicate with the new email and fee 25000 must lose.
- Dedupe **before** merge and before aggregation, otherwise counts and averages are inflated.
- `how='inner'`: unmatched keys vanish. A left join would keep course 99 with NaN names, and groupby would then silently drop it or mis-count it.
- Selecting `courses[keep]` stops extra columns appearing and avoids `_x/_y` suffix collisions.
- `.round(2)` on the net fee, and again on the average after aggregation: the spec asks for both.
- `count` on `enquiry_id` counts valid rows. It equals the number of rows because IDs have no NaN.
- `as_index=False` + `reset_index(drop=True)`: clean index.
- Nothing prints inside functions. Only the `__main__` block prints, and only the final DataFrame.

The test file to write yourself (pytest, no mocking needed)

```
import pandas as pd
import main

ENQ = "data/training_enquiries.csv"
REF = "data/training_courses.csv"


def test_load_columns_exact():
```

```
    df = main.load_student_enquiries(ENQ)
    assert list(df.columns) == ["enquiry_id", "course_id", "student_name",
                                "email", "phone", "quoted_fee",
"discount_pct"]


def test_clean_removes_missing_email_and_keeps_first_dup():
    df = main.clean_enquiry_data(main.load_student_enquiries(ENQ))
    assert df["email"].notna().all()
    assert df["enquiry_id"].is_unique
    assert df[df["enquiry_id"] == 3002].iloc[0]["email"] ==
"diya@example.com"


def test_merge_drops_unmatched():
    df = main.clean_enquiry_data(main.load_student_enquiries(ENQ))
    m = main.merge_with_courses(df, REF)
    assert 99 not in m["course_id"].values


def test_net_fee_rounding_and_no_mutation():
    df = pd.DataFrame({"quoted_fee": [1000.0], "discount_pct": [33]})
    out = main.add_net_quoted_fee(df)
    assert out["net_quoted_fee"].iloc[0] == 670.0
    assert "net_quoted_fee" not in df.columns


def test_final_shape():
    df = main.load_student_enquiries(ENQ)
    for f in (main.clean_enquiry_data,):
        df = f(df)
    df = main.merge_with_courses(df, REF)
    df = main.add_net_quoted_fee(df)
    df = main.aggregate_by_course(df)
    df = main.finalize_summary(df)
    assert list(df.columns) == ["course_name", "avg_net_quoted_fee",
"enquiry_count"]
    assert df["course_name"].tolist() == sorted(df["course_name"].tolist())
    assert list(df.index) == list(range(len(df)))
```

Variations the examiner can throw at you (practise each on the same data)

1. Keep the **last** duplicate instead of the first.
2. Also drop rows where `phone` is missing, or where `quoted_fee` is 0 or negative.
3. Treat blank-string emails as missing.
4. Use a **left** join and add a flag column `in_catalogue`.

5. Add `discount_amount = quoted_fee - net_quoted_fee`.
6. Add `savings_vs_standard = standard_fee - net_quoted_fee` (needs the merge first).
7. Aggregate with min, max, and total revenue as well as the mean.
8. Group by `category` instead of the course name, or by two keys (`category`, `course_name`).
9. Sort by `avg_net_quoted_fee` descending, ties broken by `course_name`.
10. Return only courses with at least 2 enquiries.
11. Make `finalize_summary` keep the top N rows (N as a parameter).
12. Handle an empty input DataFrame without crashing.
13. Discount given as a string like `"10%"`: clean it with `str.rstrip('%').astype(float)`.
14. Add a `discount_band` column using `pd.cut`.
15. Rename the output columns to a new spec while keeping the logic identical.

Last-minute checklist for this question

- [ ] Function names and parameter names copied exactly from the spec.
- [ ] Every function returns a DataFrame and none prints.
- [ ] Input never modified (`copy()` first).
- [ ] `keep='first'`, `how='inner'`, `subset=['email']` all written explicitly.
- [ ] Final columns in the exact order, average rounded to 2, sorted by name, clean index.
- [ ] `if __name__ == '__main__':` calls all six in order and prints only the last result.
- [ ] No hard-coded names, counts, fees or paths inside the functions.

6. FastAPI + Pydantic + SQLAlchemy question (30 marks): concepts and full solution

Exercises 3 and 4 are the same API with renamed nouns: **parent** (practice area / skill group) and **child** (consultant / trainer). Memorise the structure below once; then only swap names. I wrote this solution to your spec (Exercise 3). Note: my sandbox could not install FastAPI, so I could not execute this code here. Run the tests on your machine first and tell me any failure.

6.1 Concepts you must be able to explain

Concept	One-line meaning	Where it shows up
Pydantic model	Validates and converts incoming/outgoing data	`schemas.py`, 422 errors

Concept	One-line meaning	Where it shows up
`Field(ge=300)` / `min_length=1`	Declarative validation rules	fee, experience, names
ORM model	Python class mapped to a DB table	`models.py`
`relationship` + `ForeignKey`	Links child rows to a parent row	`back_populates` on both sides
Session + `get_db`	One DB session per request, always closed	`Depends(get_db)`
`response_model`	Shapes and filters what the API returns	nested parent output
`from_attributes=True`	Lets Pydantic read ORM objects	every `*Out` schema
`status_code=201`	Success code for creation	both POST routes
`HTTPException`	Returns 404/400 with a detail message	missing parent/child
Query parameters	Optional function args with defaults	`designation`, `sort`
TestClient + dependency override	Test without touching the real DB	`tests.py`

6.2 Status code rules from the spec (memorise)

- **422**: Pydantic validation failed (empty name, fee under 300, negative experience, floor under 1, wrong type, missing field). FastAPI does this automatically; you write no code.
- **404**: parent missing; child missing; child belongs to another parent; invalid `sort` value.
- **201**: both creates. **200**: GET and PUT.

6.3 `database.py`

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "sqlite:///./consulting.db"

engine = create_engine(DATABASE_URL, connect_args={"check_same_thread":
False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

6.4 `models.py`

```
from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import relationship
from database import Base


class PracticeArea(Base):
    __tablename__ = "practice_areas"
    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String, nullable=False)
    floor_number = Column(Integer, nullable=False)
    consultants = relationship("Consultant", back_populates="practice_area",
                               cascade="all, delete-orphan")


class Consultant(Base):
    __tablename__ = "consultants"
    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String, nullable=False)
    designation = Column(String, nullable=False)
    consultation_fee = Column(Integer, nullable=False)
    experience_years = Column(Integer, nullable=False)
    practicearea_id = Column(Integer, ForeignKey("practice_areas.id"),
nullable=False)
    practice_area = relationship("PracticeArea",
back_populates="consultants")
```

The spec says the foreign key column is named `practicearea_id` (no underscore between practice and area). Copy field names exactly.

6.5 `schemas.py`

```
from typing import List
from pydantic import BaseModel, Field, ConfigDict


class PracticeAreaCreate(BaseModel):
    name: str = Field(min_length=1)
    floor_number: int = Field(ge=1)


class ConsultantCreate(BaseModel):          # used for BOTH create and update
    name: str = Field(min_length=1)
    designation: str = Field(min_length=1)
    consultation_fee: int = Field(ge=300)
    experience_years: int = Field(ge=0)


class ConsultantOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str
    designation: str
    consultation_fee: int
    experience_years: int


class PracticeAreaOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str
    floor_number: int


class PracticeAreaWithConsultants(PracticeAreaOut):
    consultants: List[ConsultantOut] = []
```

If your course uses Pydantic v1 syntax, the equivalent is `class Config: orm_mode = True`. Use whichever your trainer's code uses; check with `pip show pydantic`.

6.6 `main.py`

```
from typing import Optional
from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session

import models, schemas
from database import engine, get_db

models.Base.metadata.create_all(bind=engine)
app = FastAPI()

SORT_FIELDS = {"name": models.Consultant.name,
               "consultation_fee": models.Consultant.consultation_fee}


@app.post("/practice-areas", response_model=schemas.PracticeAreaOut,
status_code=201)
def create_practice_area(payload: schemas.PracticeAreaCreate, db: Session =
Depends(get_db)):
    obj = models.PracticeArea(**payload.model_dump())
    db.add(obj)
    db.commit()
    db.refresh(obj)
    return obj


@app.post("/practice-areas/{parent_id}/consultants",
          response_model=schemas.ConsultantOut, status_code=201)
def create_consultant(parent_id: int, payload: schemas.ConsultantCreate,
                      db: Session = Depends(get_db)):
    parent = db.get(models.PracticeArea, parent_id)
    if parent is None:
        raise HTTPException(status_code=404, detail="Practice area not
found")
    child = models.Consultant(**payload.model_dump(),
practicearea_id=parent_id)
    db.add(child)
    db.commit()
    db.refresh(child)
    return child


@app.get("/practice-areas/{parent_id}",
response_model=schemas.PracticeAreaWithConsultants)
def get_practice_area(parent_id: int, designation: Optional[str] = None,
                      sort: Optional[str] = None, db: Session =
Depends(get_db)):
```

```
    parent = db.get(models.PracticeArea, parent_id)
    if parent is None:
        raise HTTPException(status_code=404, detail="Practice area not
found")
    if sort is not None and sort.strip() != "" and sort not in SORT_FIELDS:
        raise HTTPException(status_code=404, detail="Invalid sort field")
    q = db.query(models.Consultant).filter(models.Consultant.practicearea_id
== parent_id)
    if designation is not None and designation.strip() != "":
        q = q.filter(models.Consultant.designation == designation)
    if sort:
        q = q.order_by(SORT_FIELDS[sort])
    else:
        q = q.order_by(models.Consultant.id)
    return {"id": parent.id, "name": parent.name,
            "floor_number": parent.floor_number, "consultants": q.all()}


@app.put("/practice-areas/{parent_id}/consultants/{child_id}",
         response_model=schemas.ConsultantOut)
def update_consultant(parent_id: int, child_id: int, payload:
schemas.ConsultantCreate,
                      db: Session = Depends(get_db)):
    parent = db.get(models.PracticeArea, parent_id)
    if parent is None:
        raise HTTPException(status_code=404, detail="Practice area not
found")
    child = db.get(models.Consultant, child_id)
    if child is None or child.practicearea_id != parent_id:
        raise HTTPException(status_code=404, detail="Consultant not found")
    for k, v in payload.model_dump().items():
        setattr(child, k, v)
    db.commit()
    db.refresh(child)
    return child


if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8080)
```

Design choices to defend in a viva

- The sort field is checked against an allow-list dictionary. This avoids `getattr` on user input and gives a clean 404 for anything else. It never silently falls back.

- Parent is checked first on PUT, then the child **and** its owner. That is why a child of another parent returns 404 and is never moved.
- A blank `sort=` is treated as no sort (same spirit as the blank designation rule). If your trainer wants blank sort to be 404, change that one condition.
- `designation` is an exact, case-sensitive match. `Senior` does not match `senior`.
- The PUT replaces the four fields and never touches `practicearea_id`.

6.7 `tests.py` : isolated database, no order dependence

```
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.pool import StaticPool

from database import Base, get_db
from main import app

engine = create_engine("sqlite://", connect_args={"check_same_thread":
False},
                       poolclass=StaticPool)          # one shared in-memory
DB
TestingSession = sessionmaker(bind=engine, autoflush=False, autocommit=False)


def override_get_db():
    db = TestingSession()
    try:
        yield db
    finally:
        db.close()


app.dependency_overrides[get_db] = override_get_db
client = TestClient(app)


@pytest.fixture(autouse=True)
def fresh_db():
    Base.metadata.drop_all(bind=engine)
    Base.metadata.create_all(bind=engine)
    yield


def make_parent(name="Data", floor=3):
```

```
    return client.post("/practice-areas", json={"name": name, "floor_number":
floor}).json()


def kid(name="A", des="Senior", fee=800, exp=5):
    return {"name": name, "designation": des, "consultation_fee": fee,
"experience_years": exp}


def test_create_parent():
    r = client.post("/practice-areas", json={"name": "Data", "floor_number":
3})
    assert r.status_code == 201 and r.json()["id"] == 1


def test_create_child_and_missing_parent():
    p = make_parent()
    r = client.post(f"/practice-areas/{p['id']}/consultants", json=kid())
    assert r.status_code == 201 and r.json()["name"] == "A"
    assert client.post("/practice-areas/999/consultants",
json=kid()).status_code == 404


def test_get_nested_filter_sort():
    p = make_parent()
    for n, d, f in [("Zed", "Senior", 900), ("Amy", "Junior", 500), ("Bob",
"Senior", 700)]:
        client.post(f"/practice-areas/{p['id']}/consultants", json=kid(n, d,
f))
    base = f"/practice-areas/{p['id']}"
    assert len(client.get(base).json()["consultants"]) == 3
    seniors = client.get(base, params={"designation": "Senior"}).json()
["consultants"]
    assert {c["name"] for c in seniors} == {"Zed", "Bob"}
    names = [c["name"] for c in client.get(base, params={"sort":
"name"}).json()["consultants"]]
    assert names == ["Amy", "Bob", "Zed"]
    fees = [c["consultation_fee"] for c in
            client.get(base, params={"sort": "consultation_fee"}).json()
["consultants"]]
    assert fees == sorted(fees)
    assert client.get(base, params={"sort": "bogus"}).status_code == 404


def test_update_cases():
    p1, p2 = make_parent("A"), make_parent("B", 2)
```

```
    c = client.post(f"/practice-areas/{p1['id']}/consultants",
json=kid()).json()
    r = client.put(f"/practice-areas/{p1['id']}/consultants/{c['id']}",
json=kid("New", "Lead", 1000, 9))
    assert r.status_code == 200 and r.json()["name"] == "New"
    assert client.put(f"/practice-areas/999/consultants/{c['id']}",
json=kid()).status_code == 404
    assert client.put(f"/practice-areas/{p1['id']}/consultants/999",
json=kid()).status_code == 404
    assert client.put(f"/practice-areas/{p2['id']}/consultants/{c['id']}",
json=kid()).status_code == 404


@pytest.mark.parametrize("body", [
    {"name": "", "designation": "S", "consultation_fee": 800,
"experience_years": 1},
    {"name": "A", "designation": "", "consultation_fee": 800,
"experience_years": 1},
    {"name": "A", "designation": "S", "consultation_fee": 299,
"experience_years": 1},
    {"name": "A", "designation": "S", "consultation_fee": 800,
"experience_years": -1},
    {"name": "A", "designation": "S", "consultation_fee": "abc",
"experience_years": 1},
    {"name": "A", "designation": "S", "experience_years": 1},
])
def test_invalid_child_422(body):
    p = make_parent()
    assert client.post(f"/practice-areas/{p['id']}/consultants",
json=body).status_code == 422


def test_invalid_parent_422():
    assert client.post("/practice-areas", json={"name": "X", "floor_number":
0}).status_code == 422
    assert client.post("/practice-areas", json={"name": "", "floor_number":
1}).status_code == 422
```

6.8 FastAPI variations the examiner can throw at you

1. Add `DELETE /practice-areas/{parent_id}/consultants/{child_id}` returning 204, with the same ownership check.
2. Add `min_fee` / `max_fee` query filters (`>=`, `<=`), or `min_experience`.
3. Add `order=desc` for sorting, with 404 for any other value.

4. Add pagination: `skip` and `limit` (limit between 1 and 100, otherwise 422 using `Query(ge=1, le=100)`).
5. Add `GET /practice-areas` listing all parents, with a count of consultants.
6. Duplicate-name rule: return 409 if a parent with the same name exists.
7. Make `designation` filter case-insensitive (`func.lower`).
8. Add `PATCH` for partial update using `Optional` fields and `model_dump(exclude_unset=True)`.
9. Return 404 when a filter matches no children (and know why that is usually the wrong design).
10. Add `POST /practice-areas/{id}/consultants/bulk` taking a list of consultants.
11. Enforce a unique consultant name within one parent.
12. Switch the database file name or table names (read the spec's names carefully).
13. Add a custom 404 JSON error shape using an exception handler.
14. Add a simple middleware that adds an `X-Process-Time` header; add CORS for a given origin.
15. Add an API-key header dependency; missing key returns 401.

Where people lose marks in FastAPI questions

- Forgetting `status_code=201` on the POST decorator (the default is 200).
- Wrong field names: `practicearea_id` vs `practice_area_id`, `consultation_fee` vs `fee`.
- `response_model` missing, so the nested list or the ORM object does not serialise.
- Missing `from_attributes=True` (or `orm_mode`), causing a validation error on the response.
- Forgetting `db.commit()` or `db.refresh()`, so the generated `id` is `None`.
- Not closing the session (`finally: db.close()`).
- Running `uvicorn.run(...)` at module level, so importing in tests starts the server.
- Letting 422 happen for a missing parent because the body check runs first. That is correct FastAPI behaviour: body validation happens before your code. In your own tests, send a **valid** body when you test a 404.
- Changing the child's parent in PUT. The spec forbids it.
- Returning 400 or 200 with an error message instead of raising `HTTPException(404)`.
- Tests that share a database and depend on order. Reset tables per test.
- Hard-coding records inside the endpoints.
- Using `class Config` and `model_config` together, or mixing Pydantic v1 and v2 syntax.

- Typing the path parameter as `str` instead of `int`; a non-integer then gives 422, not 404.

7. Error messages, final checklists and a mock exam

7.1 Error messages you will see and what they mean

Message	Cause	Fix
`KeyError: 'col'`	Column name typo or the column was dropped earlier	Print `df.columns.tolist()`
`The truth value of a Series/array is ambiguous`	Used `and` / `or` / `if` on a Series	Use `&`, `|`, `.any()`, `.all()`
`SettingWithCopyWarning`	Assigned into a slice	`.copy()` first, or `.loc[mask, col] = v`
`ValueError: You are trying to merge on object and int64 columns`	Key dtypes differ	`astype(int)` on both keys
`ValueError: shapes (a,b) and (c,d) not aligned`	Bad matrix multiplication shapes	Check `.shape`, transpose
`AttributeError: 'NoneType' ...`	A function forgot `return`	Add `return`
`TypeError: Object of type X is not JSON serializable`	Returned an ORM object without a response model	Set `response_model`
`sqlite3.OperationalError: no such table`	Tables not created, or wrong DB file	`Base.metadata.create_all` on import
`sqlalchemy.exc.InvalidRequestError ... mapper`	Relationship names or `back_populates` mismatch	Match both sides exactly
`422 Unprocessable Entity`	Body failed Pydantic rules	Read the `detail` list

Message	Cause	Fix
`405 Method Not Allowed`	Wrong HTTP verb or path	Compare decorator and test call
`ImportError: cannot import name`	Circular import or wrong file name	Keep `database`, `models`, `schemas` independent of `main`

7.2 Exact-output habits that save marks

- Copy-paste names from the spec into your code. Do not retype them.
- After each function, print `df.columns.tolist()` and `df.index` in a scratch session.
- Count by hand on the sample data and compare with your output. Your hand-count is the real test.
- For APIs, send one valid request and one invalid request per rule using Swagger at `/docs`, then write the pytest version.
- Never leave `print` / `breakpoint()` or commented experiments in the final file.
- Re-run the whole file from a fresh terminal to catch missing imports.

7.3 Mixed NumPy + Pandas mock question (45 minutes, try it cold)

You get `students.csv` with columns `student_id, name, maths, physics, chemistry, attendance_pct`. Some marks are missing, and `student_id` has duplicates.

1. `load_students(path)` returns the CSV as a DataFrame.
2. `clean_students(df)` drops duplicate IDs (keep first), fills missing marks with that subject's mean, and keeps attendance between 0 and 100.
3. `add_scores(df)` adds `total` and `average` (rounded to 2) using NumPy or Pandas vectorised operations.
4. `add_grade(df)` uses `np.select`: average at least 75 is `A`, at least 50 is `B`, else `C`.
5. `top_students(df, n)` returns the top `n` by average with a clean index and exact column order.
6. Using NumPy only: return the z-score array of `average`, and the count of students whose z-score is above 1.

Check yourself: no loops, no input modification, no printing, clean index, `n` larger than the number of rows must not crash.

7.4 Readiness test: you are exam-ready when you can do all of these without looking

- [ ] Write the six-function Pandas pipeline from a blank file in under 25 minutes.
- [ ] Write database.py, models.py, schemas.py and main.py for a new parent/child pair in under 40 minutes.

- [ ] Write 8 pytest tests for the API, including the three 404 update cases and the 422 cases.
- [ ] Explain inner vs left join, `keep='first'` vs `'last'`, `axis=0` vs `axis=1`, `&` vs `and`, 404 vs 422, `commit` vs `refresh`.
- [ ] Solve the 15 NumPy questions and 12 Pandas questions in Sections 3 and 4 on a laptop with no notes.
- [ ] Fix a deliberately broken version of each solution (change one thing in my code and find it).

7.5 A simple rhythm for the days before the milestone

You have about five days. Do not read; type. Pass 1: type Sections 5 and 6 from this guide until the tests pass. Pass 2: rewrite them from the spec only. Pass 3: solve the variation lists (5.x and 6.8) and the mock question above under a timer. Send me anything that fails and I will explain the cause and make you a fresh question on that exact weak spot.